



ESCUELA POLITÉCNICA NACIONAL

ESCUELA DE FORMACIÓN DE TECNÓLOGOS



BASE DE DATOS

ASIGNATURA:

BASE DE DATOS

PROFESOR:

Ing. Yadira Gissel Franco Guevara

PERÍODO ACADÉMICO:

2025-B

PROYECTO GRUPAL

TÍTULO

BASE DE DATOS - FLOTA DE BUSES

PROYECTO SEGUNDO BIMESTRE

ESTUDIANTES:

Edwin Efraín Chusin Enríquez

Ismael Sebastián Narvaez Curillo

Tabla de contenido

1. INTRODUCCION Y OBJETIVOS	4
1.1 Descripción general del proyecto.....	4
1.2 Problema a resolver	4
1.3 Objetivos específicos	5
1.4 Alcance del sistema	6
2. MODELO CONCEPTUAL (Diagrama ER).....	6
2.1 Descripción general de entidades.....	6
2.2 Diagrama Entidad-Relación (imagen)	7
2.3 Descripción de relaciones y cardinalidades	8
2.4 Normalización (Forma Normal)	9
3. MODELO LÓGICO	9
3.1 Esquema de la base de datos	9
3.2 Diccionario de Datos (tabla por tabla).....	10
Tabla: pasajeros	10
Tabla: conductores.....	10
Tabla: buses.....	10
Tabla: rutas	10
Tabla: asignaciones	11
Tabla: boletos	11
Tabla: auditoria	11
3.3 Índices creados.....	12
4. IMPLEMENTACIÓN EN EL SGBD.....	12
4.1 Capturas de ejecución en MySQL	12
Creación de base de datos	12
Creación de tablas.....	13
Inserción de datos válidos	13
Intentos con datos inválidos (pruebas de constraints)	14
4.2 Población de datos iniciales	16
5. CONSULTAS Y RESULTADOS	17

5.1 Consultas básicas SELECT.....	17
5.2 Consultas con JOINS	18
5.3 Consultas con agregaciones (SUM, COUNT, AVG).....	19
5.4 Consultas con funciones de cadena	19
5.5 Subconsultas.....	20
5.6 Optimización de consultas (EXPLAIN, EXPLAIN ANALYZE)	20
Conclusiones sobre índices	21
6. PRUEBAS	21
Pruebas de rendimiento	21
Prueba de carga.....	22
Prueba de Estrés.....	23
7. BACKUPS	23
BuckUp completo	23
Buck Up en caliente	24
8. CONCLUSIONES SOBRE LO APRENDIDO.....	26
8.1 Logros alcanzados.....	26
8.2 Desafíos encontrados y solucionados.....	27
8.3 Lecciones aprendidas	27
CONCLUSION GENERAL	28
9. LINKS DE ENTREGA.....	28

1. INTRODUCCION Y OBJETIVOS

1.1 Descripción general del proyecto

El presente proyecto corresponde al desarrollo de un Sistema de Gestión de Flota de Buses para empresas transportistas cantonales de la provincia de Pichincha, Ecuador. Este sistema está diseñado para modernizar y optimizar los procesos de operación, control y administración de flotas de transporte urbano e interurbano.

Actualmente, las empresas transportistas cantonales operan con métodos manuales y desorganizados que generan ineficiencias, pérdida de información y falta de trazabilidad en sus operaciones. El sistema propuesto implementa una solución integral basada en una base de datos relacional que permite registrar, gestionar y consultar toda la información relacionada con:

Pasajeros: Registro centralizado de usuarios del servicio

Conductores: Información y asignación de personal operativo

Buses: Inventario y control de vehículos

Rutas: Trayectos disponibles, horarios y paradas

Boletos: Venta de pasajes y control de ingresos

Auditoría: Registro histórico de todas las operaciones para control y trazabilidad

El sistema fue desarrollado utilizando MySQL como Sistema Gestor de Base de Datos (SGBD) y PHP como lenguaje backend, integrándose con una interfaz web moderna para acceso desde cualquier oficina de la empresa.

1.2 Problema a resolver

En la actualidad, las empresas transportistas cantonales de Pichincha enfrentan los siguientes problemas operacionales:

Desorganización en control de rutas y horarios: Las rutas se controlan mediante comunicaciones por radio o mensajes de texto entre conductores y despachadores. No existe registro oficial de horarios de salida y llegada, y es imposible auditar el cumplimiento de horarios.

Falta de control sobre pasajeros: Los pasajeros se registran manualmente en hojas de papel (formularios físicos) que firman en el autobús. Estas hojas se pierden o deterioran, y en caso de emergencia no hay forma rápida de localizar a los pasajeros. No existe histórico de viajes realizados.

Gestión deficiente de venta de boletos: Los boletos se venden de forma manual sin control sobre asientos ocupados. Existe riesgo de sobreventa, los precios varían

según el conductor, no hay comprobantes digitales, e imposible generar reportes de ingresos.

Ausencia de información sobre conductores: No hay asignación formal de conductores a rutas y desconocimiento sobre cuál conductor viajó en qué fecha y ruta.

Falta de auditoría y trazabilidad: No existe registro de cambios en la información, imposible detectar inconsistencias o fraudes, y no hay comprobación ante reclamos de pasajeros.

Dificultad en toma de decisiones: Los gerentes no tienen datos confiables para análisis. Las decisiones se toman basadas en estimaciones, no en datos reales.

1.3 Objetivos específicos

Objetivo General

Desarrollar un Sistema de Gestión Integral de Flota de Buses que proporcione control, automatización y trazabilidad total de las operaciones de empresas transportistas cantonales de Pichincha.

Objetivos Específicos:

- Digitalizar el registro de pasajeros reemplazando formularios en papel con un registro digital centralizado.
- Automatizar la gestión de rutas y horarios con validación de tiempos y auditoría
- Optimizar la venta de boletos implementando sistema automatizado con prevención de sobreventa
- Gestionar la asignación formal de conductores a rutas con auditoría de responsabilidades
- Implementar auditoría completa registrando todas las operaciones (INSERT, UPDATE, DELETE)
- Facilitar análisis y toma de decisiones generando reportes de ingresos, ocupación y rendimiento
- Implementar seguridad mediante roles de usuario y protección contra inyecciones SQL
- Optimizar rendimiento mediante índices y consultas eficientes

1.4 Alcance del sistema

¿QUÉ SÍ INCLUYE?

- Gestión de pasajeros con registro digital centralizado y validación de email único
- Gestión de buses con control de capacidad (1-80 asientos) y placas únicas
- Gestión de rutas con validación de horarios (hora llegada > hora salida)
- Venta de boletos con validaciones de disponibilidad de asientos
- Auditoría completa de todas las operaciones como de ingresos por semana, ruta y período
- Control de acceso por roles (Operador y Administrador)
- Protección contra SQL injection mediante prepared statements

¿QUÉ NO INCLUYE?

- Sistema de pagos en línea (la compra es presencial)
- Aplicación móvil nativa (solo versión web responsive)
- Sistema de multas o penalizaciones a conductores
- Gestión de mantenimiento de buses

2. MODELO CONCEPTUAL (Diagrama ER)

2.1 Descripción general de entidades

El modelo conceptual del sistema de gestión de flota de buses está compuesto por 7 entidades principales que representan los elementos clave del negocio:

PASAJEROS: Entidad que almacena la información de los usuarios del servicio de transporte. Cada pasajero es identificado de forma única mediante un ID autogenerado. Se registran datos personales como nombre, apellido, email (único) y teléfono para contacto.

CONDUCTORES: Entidad que contiene la información del personal operativo. Cada conductor tiene un ID único, nombre completo, cédula de identidad (única) y número de teléfono para comunicaciones urgentes.

BUSES: Entidad que representa los vehículos de la flota. Cada bus tiene un ID único, modelo del vehículo, placa (única y obligatoria), año de fabricación y capacidad en número de asientos.

RUTAS: Entidad que define los trayectos disponibles. Cada ruta tiene un ID único, punto de origen, punto de destino, hora de salida programada y hora de llegada esperada.

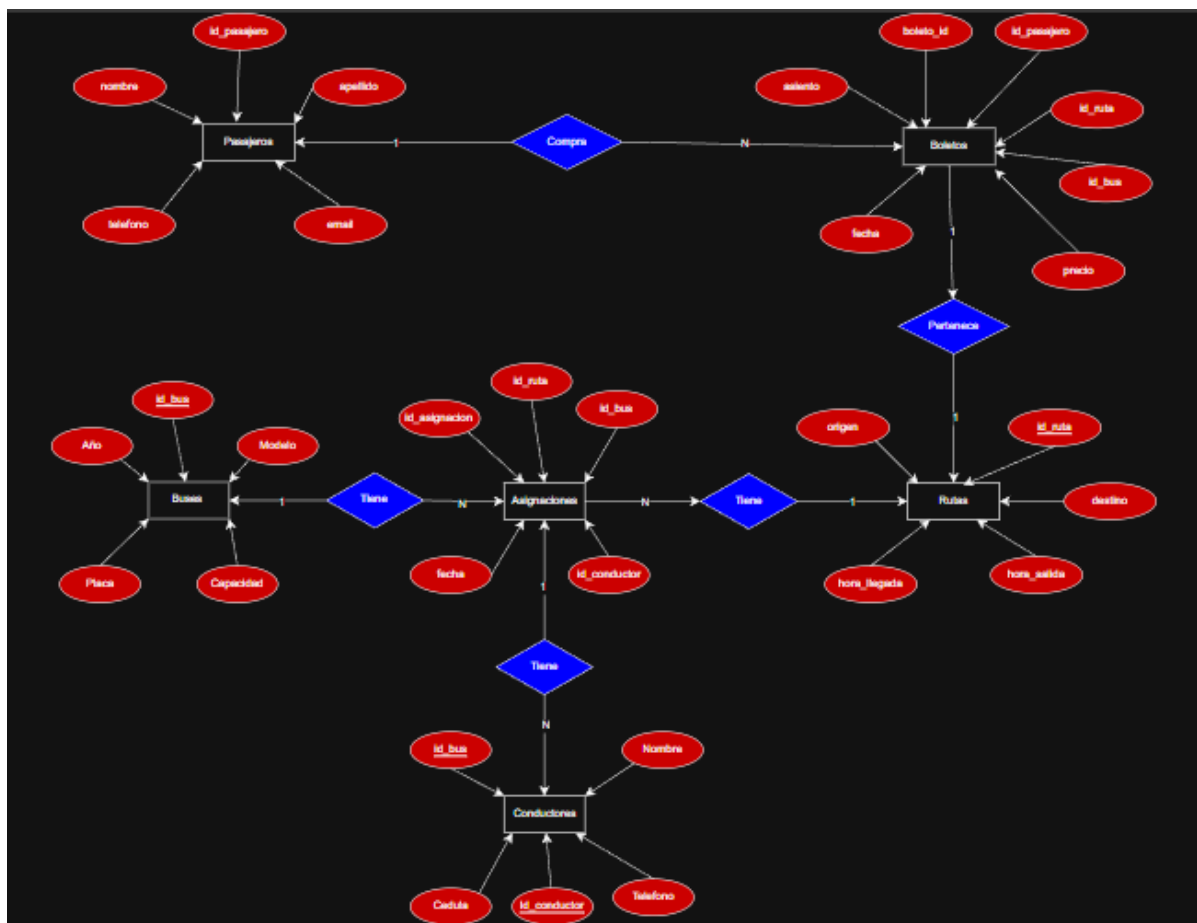
ASIGNACIONES: Entidad que relaciona conductores, buses y rutas. Registra qué conductor operará qué bus en qué ruta y en qué fecha específica.

BOLETOS: Entidad que representa la venta de pasajes. Cada boleto vincula un pasajero a una ruta, registra el asiento asignado, el precio pagado, la fecha de venta y el bus en el cual viajará.

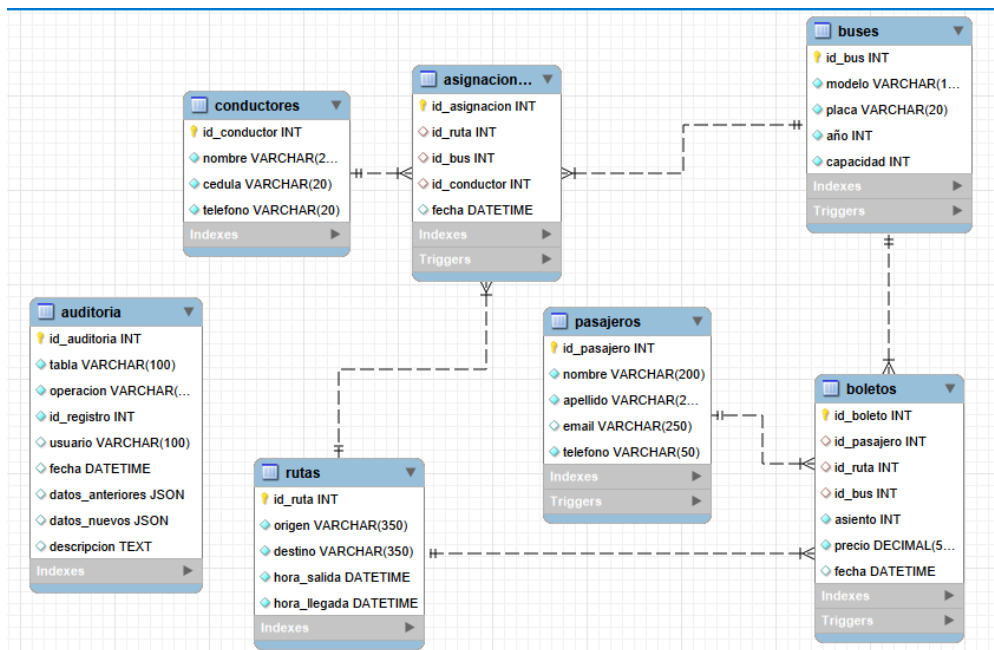
AUDITORIA: Entidad que mantiene un registro histórico de todos los cambios en el sistema. Registra qué tabla fue modificada, qué operación se realizó (INSERT, UPDATE, DELETE), quién la realizó, cuándo ocurrió, y almacena los datos anteriores y nuevos para trazabilidad completa.

2.2 Diagrama Entidad-Relación (imagen)

La imagen del modelo E-R de nuestra BD flota_de_buses:



Para un mejor entendimiento también presentamos el diagrama generado por reverse – engineer de mysql workbench:



2.3 Descripción de relaciones y cardinalidades

Relación: PASAJEROS - BOLETOS

- Cardinalidad: 1:N (Uno a Muchos)
- Descripción: Un pasajero puede comprar múltiples boletos en diferentes fechas y rutas. Un boleto pertenece a exactamente un pasajero.
- Restricción: ON DELETE RESTRICT (no se puede eliminar un pasajero que tiene boletos), ON UPDATE CASCADE (si cambia el ID del pasajero, se actualizan los boletos)

Relación: RUTAS - BOLETOS

- Cardinalidad: 1:N (Uno a Muchos)
- Descripción: Una ruta puede tener múltiples boletos vendidos. Un boleto corresponde a exactamente una ruta.
- Restricción: ON DELETE RESTRICT, ON UPDATE CASCADE

Relación: BUSES - BOLETOS

- Cardinalidad: 1:N (Uno a Muchos)
- Descripción: Un bus puede tener múltiples boletos asignados en diferentes viajes. Un boleto especifica en qué bus viaja el pasajero.
- Restricción: ON DELETE RESTRICT, ON UPDATE CASCADE

Relación: RUTAS - ASIGNACIONES

- Cardinalidad: 1:N (Uno a Muchos)
- Descripción: Una ruta puede tener múltiples asignaciones a lo largo del tiempo. Una asignación corresponde a una ruta específica.

- Restricción: ON DELETE RESTRICT, ON UPDATE CASCADE

Relación: BUSES - ASIGNACIONES

- Cardinalidad: 1:N (Uno a Muchos)
- Descripción: Un bus puede ser asignado a múltiples rutas en diferentes fechas. Una asignación especifica qué bus se usa.
- Restricción: ON DELETE RESTRICT, ON UPDATE CASCADE

Relación: CONDUCTORES - ASIGNACIONES

- Cardinalidad: 1:N (Uno a Muchos)
- Descripción: Un conductor puede tener múltiples asignaciones a diferentes rutas. Una asignación especifica qué conductor la realiza.
- Restricción: ON DELETE RESTRICT, ON UPDATE CASCADE

2.4 Normalización (Forma Normal)

El modelo de datos implementado se encuentra normalizado hasta la Tercera Forma Normal (3FN):

Primera Forma Normal (1FN): Cumple porque cada atributo contiene valores atómicos (no hay repeating groups). Por ejemplo, cada pasajero tiene un solo nombre, un solo apellido, un solo email, no hay listas de valores en un solo campo.

Segunda Forma Normal (2FN): Cumple porque todos los atributos dependen completamente de la clave primaria. Ejemplo en tabla BOLETOS: el precio depende del boleto completo, no solo de parte de la clave.

Tercera Forma Normal (3FN): Cumple porque no existen dependencias transitivas. Cada atributo depende solo de la clave primaria, no de otros atributos no clave. Por ejemplo, en BUSES no almacenamos el nombre del modelo derivado del año, cada dato es independiente.

Conclusión de Normalización: El diseño evita redundancia, garantiza integridad de datos y facilita mantenimiento futuro del sistema.

3. MODELO LÓGICO

3.1 Esquema de la base de datos

`create database flota_de_buses;`

`use flota_de_buses;` → Para crear y usar la base de datos principal.

La base de datos **flota_de_buses** contiene 7 tablas principales que implementan el modelo lógico con las siguientes características:

- Claves primarias autogeneradas
- Validaciones mediante CHECK constraints
- Unicidad en campos críticos (email, cédula, placa)
- Integridad referencial mediante Foreign Keys
- Auditoría automática mediante triggers

3.2 Diccionario de Datos (tabla por tabla)

Tabla: pasajeros

```
create table pasajeros(
    id_pasajero int primary key auto_increment,
    nombre varchar(200) not null,
    apellido varchar(200) not null,
    email varchar (250) unique,
    telefono varchar (50) not null
);
```

Tabla: conductores

```
create table conductores(
    id_conductor int primary key auto_increment,
    nombre varchar(200) not null,
    cedula varchar(20) not null unique,
    telefono varchar(20) not null
);
```

Tabla: buses

```
create table buses(
    id_bus int primary key auto_increment,
    modelo varchar (100) not null,
    placa varchar (20) not null unique,
    año int not null check(año > 2000),
    capacidad int not null check(capacidad between 1 and 80)
);
```

Tabla: rutas

```
create table rutas(
    id_ruta int primary key auto_increment,
    origen varchar(350) not null,
    destino varchar(350) not null,
    hora_salida datetime default current_timestamp not null,
    hora_llegada datetime not null
);
```

);

Tabla: asignaciones

```
create table asignaciones(  
    id_asignacion int primary key auto_increment,  
    id_ruta int,  
    id_bus int,  
    id_conductor int,  
    fecha datetime default current_timestamp,  
    foreign key (id_ruta) references rutas(id_ruta),  
    foreign key (id_bus) references buses(id_bus),  
    foreign key (id_conductor) references conductores(id_conductor)  
);
```

Tabla: boletos

```
create table boletos(  
    id boleto int primary key auto_increment,  
    id_pasajero int,  
    id_ruta int,  
    id_bus int,  
    asiento int check(asiento between 1 and 60) not null,  
    precio decimal(5,2) check(precio between 1 and 100.00) not null,  
    fecha datetime default current_timestamp,  
    foreign key (id_pasajero) references pasajeros(id_pasajero),  
    foreign key (id_ruta) references rutas(id_ruta),  
    foreign key (id_bus) references buses(id_bus)  
);
```

Tabla: auditoria

```
create table auditoria (  
    id_auditoria int primary key auto_increment,  
    tabla varchar(100) not null,  
    operacion varchar(10) not null,  
    id_registro int not null,  
    usuario varchar(100),  
    fecha datetime default current_timestamp,  
    datos_anteriores JSON,  
    datos_nuevos JSON,  
    descripcion text  
);
```

3.3 Índices creados

```
create index idx_pasajero_email on pasajeros(email);
create index idx_conductor_cedula on conductores(cedula);
create index idx_bus_placa on buses(placa);
create index idx_ruta_origen_destino on rutas( origen, destino);
create index idx_boleto_pasajero on boletos(id_pasajero);
create index idx_boleto_ruta on boletos(id_ruta);
create index idx_boleto_bus on boletos(id_bus);
create index idx_boleto_fecha on boletos(fecha);
create index idx_asignacion_ruta on asignaciones(id_ruta);
create index idx_asignacion_bus on asignaciones(id_bus);
create index idx_asignacion_conductor on asignaciones(id_conductor);
CREATE INDEX idx_boletos_ruta_bus ON boletos(id_ruta, id_bus);
CREATE INDEX idx_asignaciones_ruta_conductor ON asignaciones(id_ruta,
id_conductor);
CREATE INDEX idx_asignaciones_bus_fecha ON asignaciones(id_bus, fecha);
```

4. IMPLEMENTACIÓN EN EL SGBD

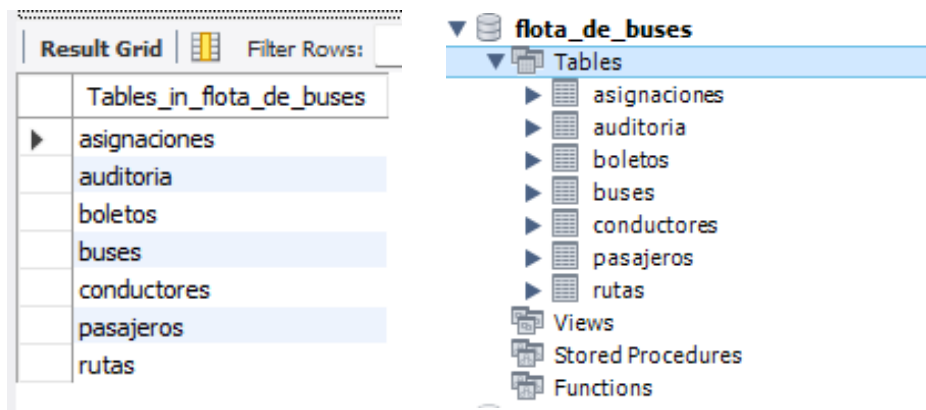
4.1 Capturas de ejecución en MySQL

Creación de base de datos

✓	2	21:51:51	create database flota_de_buses
✓	3	21:51:51	use flota_de_buses



Creación de tablas



Inserción de datos válidos

-- DATOS VALIDOS PARA POBLAR LA BASE DE DATOS CON REGISTROS INICIALES

INSERT INTO buses (modelo, placa, año, capacidad) VALUES

('Mercedes-Benz OF1721', 'PBA-1234', 2015, 60),

('Hino FC9J', 'PBC-5678', 2018, 45),

('Chevrolet NPR', 'PBD-9012', 2021, 50);

INSERT INTO buses (modelo, placa, año, capacidad) VALUES

('Scania K400', 'PBE-1122', 2022, 60),

('Volvo B11R', 'PBF-3344', 2023, 55),

('Yutong ZK6129H', 'PBG-5566', 2019, 42),

('Marcopolo Paradiso 1200', 'PBH-7788', 2020, 58),

('Volkswagen 17.230', 'PBI-9900', 2017, 48);

INSERT INTO conductores (nombre, cedula, telefono) VALUES

('Carlos Pérez', '1723456789', '0998765432'),

('María López', '1712345678', '0987654321'),

('José Andrade', '1756789012', '0976543210');

INSERT INTO rutas (origen, destino, hora_salida, hora_llegada) VALUES

('Quito', 'Cayambe', '2026-01-22 07:00:00', '2026-01-22 09:00:00'),

('Quito', 'Mejía (Machachi)', '2026-01-22 08:30:00', '2026-01-22 10:00:00'),

```
('Quito', 'Pedro Moncayo (Tabacundo)', '2026-01-22 06:00:00', '2026-01-22 08:00:00');
```

```
INSERT INTO rutas (origen, destino, hora_salida, hora_llegada) VALUES
```

```
('Quito', 'Rumiñahui (Sangolquí)', '2026-01-22 09:00:00', '2026-01-22 10:15:00'),
```

```
('Quito', 'Ibarra', '2026-01-22 10:00:00', '2026-01-22 13:00:00'),
```

```
('Quito', 'Latacunga', '2026-01-22 11:30:00', '2026-01-22 13:30:00'),
```

```
('Quito', 'Ambato', '2026-01-22 13:00:00', '2026-01-22 16:00:00'),
```

```
('Quito', 'Santo Domingo', '2026-01-22 14:00:00', '2026-01-22 17:30:00');
```

```
INSERT INTO pasajeros (nombre, apellido, email, telefono) VALUES
```

```
('Ana', 'Guamán', 'ana.guaman@example.com', '0991122334'),
```

```
('Luis', 'Chávez', 'luis.chavez@example.com', '0989988776'),
```

```
('Sofía', 'Mora', 'sofia.mora@example.com', '0977766554');
```

```
INSERT INTO boletos (id_pasajero, id_ruta, id_bus, asiento, precio, fecha) VALUES
```

```
(1, 1, 1, 12, 2.50, '2026-01-22 07:05:00'),
```

```
(2, 2, 2, 20, 1.75, '2026-01-22 08:35:00'),
```

```
(3, 3, 3, 5, 3.00, '2026-01-22 06:10:00');
```

```
INSERT INTO asignaciones (id_ruta, id_bus, id_conductor, fecha) VALUES
```

```
(1, 1, 1, '2026-01-22 06:30:00'),
```

```
(2, 2, 2, '2026-01-22 08:00:00'),
```

```
(3, 3, 3, '2026-01-22 05:30:00');
```

Intentos con datos inválidos (pruebas de constraints)

```
-- DATOS NO VALIDOS QUE COMPRUEBAN SI LAS REGLAS DE LAS TABLAS CUMPLEN CON LA LOGICA DE NEGOCIO
```

```
-- Año menor a 2000 (violación CHECK)
```

```
INSERT INTO buses (modelo, placa, año, capacidad) VALUES
```

```
('Volvo B7R', 'PBE-3456', 1999, 55);
```

-- Capacidad fuera de rango (>80)

```
INSERT INTO buses (modelo, placa, año, capacidad) VALUES  
('Scania K310', 'PBF-7890', 2017, 100);
```

-- Placa duplicada (violación UNIQUE)

```
INSERT INTO buses (modelo, placa, año, capacidad) VALUES  
('Mercedes-Benz OF1721', 'PBA-1234', 2019, 60);
```

-- Cédula duplicada (violación UNIQUE)

```
INSERT INTO conductores (nombre, cedula, telefono) VALUES  
('Pedro Salazar', '1723456789', '0991112223');
```

-- Hora de llegada anterior a la salida (inconsistencia lógica)

```
INSERT INTO rutas (origen, destino, hora_salida, hora_llegada) VALUES  
('Quito', 'Rumiñahui (Sangolquí)', '2026-01-22 09:00:00', '2026-01-22 08:00:00');
```

-- Asiento fuera de rango (>60)

```
INSERT INTO boletos (id_pasajero, id_ruta, id_bus, asiento, precio, fecha) VALUES  
(1, 1, 1, 75, 2.50, '2026-01-22 07:10:00');
```

-- Precio fuera de rango (<1)

```
INSERT INTO boletos (id_pasajero, id_ruta, id_bus, asiento, precio, fecha) VALUES  
(2, 2, 2, 15, 0.50, '2026-01-22 08:40:00');
```

4.2 Población de datos iniciales

	id_conductor	nombre	cedula	telefono
▶	1	Carlos Pérez	1723456789	0998765432
	2	María López	1712345678	0987654321
	3	José Andrade	José Andrade	0976543210
*	NULL	NULL	NULL	NULL

	id_pasajero	nombre	apellido	email	telefono
▶	1	Ana	Guamán	ana.guaman@example.com	0991122334
	2	Luis	Chávez	luis.chavez@example.com	0989988776
	3	Sofía	Mora	sofia.mora@example.com	0977766554
*	NULL	NULL	NULL	NULL	NULL

	id_asignacion	id_ruta	id_bus	id_conductor	fecha
▶	1	1	1	1	2026-01-22 06:30:00
	2	2	2	2	2026-01-22 08:00:00
	3	3	3	3	2026-01-22 05:30:00
*	NULL	NULL	NULL	NULL	NULL

	id_boleto	id_pasajero	id_ruta	id_bus	asiento	precio	fecha
▶	1	1	1	1	12	2.50	2026-01-22 07:05:00
	2	2	2	2	20	1.75	2026-01-22 08:35:00
	3	3	3	3	5	3.00	2026-01-22 06:10:00
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

	id_bus	modelo	placa	año	capacidad
▶	1	Mercedes-Benz OF1721	PBA-1234	2015	60
	2	Hino FC9J	PBC-5678	2018	45
	3	Chevrolet NPR	PBD-9012	2021	50
	4	Scania K400	PBE-1122	2022	60
	5	Volvo B11R	PBF-3344	2023	55
	6	Yutong ZK6129H	PBG-5566	2019	42
	7	Marcopolo Paradiso 1200	PBH-7788	2020	58
	8	Volkswagen 17.230	PBI-9900	2017	48
*	NULL	NULL	NULL	NULL	NULL

	id_ruta	origen	destino	hora_salida	hora_llegada
▶	1	Quito	Cayambe	2026-01-22 07:00:00	2026-01-22 09:00:00
	2	Quito	Mejía (Machachi)	2026-01-22 08:30:00	2026-01-22 10:00:00
	3	Quito	Pedro Moncayo (Tabacundo)	2026-01-22 06:00:00	2026-01-22 08:00:00
	4	Quito	Rumiñahui (Sangolquí)	2026-01-22 09:00:00	2026-01-22 10:15:00
	5	Quito	Ibarra	2026-01-22 10:00:00	2026-01-22 13:00:00
	6	Quito	Latacunga	2026-01-22 11:30:00	2026-01-22 13:30:00
	7	Quito	Ambato	2026-01-22 13:00:00	2026-01-22 16:00:00
	8	Quito	Santo Domingo	2026-01-22 14:00:00	2026-01-22 17:30:00
*	NULL	NULL	NULL	NULL	NULL

5. CONSULTAS Y RESULTADOS

5.1 Consultas básicas SELECT

CONSULTA + CAPTURA DE LOS RESULTADOS

listar todos los buses disponibles

	id_bus	modelo	placa	año	capacidad
▶	1	Mercedes-Benz OF1721	PBA-1234	2015	60
	2	Hino FC9J	PBC-5678	2018	45
	3	Chevrolet NPR	PBD-9012	2021	50
	4	Scania K400	PBE-1122	2022	60
	5	Volvo B11R	PBF-3344	2023	55
	6	Yutong ZK6129H	PBG-5566	2019	42
	7	Marcopolo Paradiso 1200	PBH-7788	2020	58
	8	Volkswagen 17.230	PBI-9900	2017	48
	NULL	NULL	NULL	NULL	NULL

pasajeros que tienen email registrado

	nombre	apellido	email
▶	Ana	Guamán	ana.guaman@example.com
	Luis	Chávez	luis.chavez@example.com
	Sofía	Mora	sofia.mora@example.com

rutas que salen desde quito

	id_ruta	origen	destino	hora_salida	hora_llegada
▶	1	Quito	Cayambe	2026-01-22 07:00:00	2026-01-22 09:00:00
	2	Quito	Mejía (Machachi)	2026-01-22 08:30:00	2026-01-22 10:00:00
	3	Quito	Pedro Moncayo (Tabacundo)	2026-01-22 06:00:00	2026-01-22 08:00:00
	4	Quito	Rumiñahui (Sangolquí)	2026-01-22 09:00:00	2026-01-22 10:15:00
	5	Quito	Ibarra	2026-01-22 10:00:00	2026-01-22 13:00:00
	6	Quito	Latacunga	2026-01-22 11:30:00	2026-01-22 13:30:00
	7	Quito	Ambato	2026-01-22 13:00:00	2026-01-22 16:00:00
	8	Quito	Santo Domingo	2026-01-22 14:00:00	2026-01-22 17:30:00
	NULL	NULL	NULL	NULL	NULL

buses con capacidad mayor a 50

Result Grid			
	modelo	placa	capacidad
▶	Mercedes-Benz OF1721	PBA-1234	60
	Scania K400	PBE-1122	60
	Volvo B11R	PBF-3344	55
	Marcopolo Paradiso 1200	PBH-7788	58

5.2 Consultas con JOINS

CONSULTA + CAPTURA DE LOS RESULTADOS

boletos con datos del pasajero

Result Grid				Filter Rows:
	nombre	apellido	id_boleto	precio
▶	Ana	Guamán	1	2.50
	Luis	Chávez	2	1.75
	Sofía	Mora	3	3.00

boletos con origen y destino de la ruta

	id_boleto	origen	destino	precio
▶	1	Quito	Cayambe	2.50
	2	Quito	Mejía (Machachi)	1.75
	3	Quito	Pedro Moncayo (Tabacundo)	3.00

asignaciones con bus y conductor

	id_asignacion	placa	nombre
▶	1	PBA-1234	Carlos Pérez
	2	PBC-5678	María López
	3	PBD-9012	José Andrade

boletos con bus asignado

	id_boleto	placa	asiento
▶	1	PBA-1234	12
	2	PBC-5678	20
	3	PBD-9012	5

rutas con buses asignados

	origen	destino	placa
▶	Quito	Cayambe	PBA-1234
	Quito	Mejía (Machachi)	PBC-5678
	Quito	Pedro Moncayo (Tabacundo)	PBD-9012

pasajeros y rutas que han utilizado

	nombre	apellido	origen	destino
▶	Ana	Guamán	Quito	Cayambe
	Luis	Chávez	Quito	Mejía (Machachi)
	Sofía	Mora	Quito	Pedro Moncayo (Tabacundo)

5.3 Consultas con agregaciones (SUM, COUNT, AVG)

CONSULTA + CAPTURA DE LOS RESULTADOS

total de ingresos por semana

	semana	total_ingresos
▶	3	7.25

cantidad de boletos vendidos por ruta

	origen	destino	total_boletos
▶	Quito	Cayambe	1
	Quito	Mejía (Machachi)	1
	Quito	Pedro Moncayo (Tabacundo)	1

promedio de precio de boletos por bus

	placa	precio_promedio
▶	PBA-1234	2.500000
	PBC-5678	1.750000
	PBD-9012	3.000000

5.4 Consultas con funciones de cadena

CONSULTA + CAPTURA DE LOS RESULTADOS

nombre completo del pasajero

	nombre_completo
▶	Ana Guamán
	Luis Chávez
	Sofía Mora

convertir nombres de conductores a mayúsculas

	nombre_mayuscula
▶	CARLOS PÉREZ
	MARÍA LÓPEZ
	JOSÉ ANDRADE

concatena nombre y apellido y genera una descripción legible del boleto

	pasajero	descripcion_boleto
▶	Ana Guamán	boleto para la ruta Quito - Cayambe asiento 12
	Luis Chávez	boleto para la ruta Quito - Mejía (Machachi) asie...
	Sofía Mora	boleto para la ruta Quito - Pedro Moncayo (Tab...

5.5 Subconsultas

CONSULTA + CAPTURA DE LOS RESULTADOS

pasajeros que han comprado boletos

	id_pasajero	nombre	apellido	email	telefono
▶	1	Ana	Guamán	ana.guaman@example.com	0991122334
	2	Luis	Chávez	luis.chavez@example.com	0989988776
	3	Sofía	Mora	sofia.mora@example.com	0977766554
*	NULL	NULL	NULL	NULL	NULL

buses que no tienen boletos vendidos

	id_bus	modelo	placa	año	capacidad
▶	4	Scania K400	PBE-1122	2022	60
	5	Volvo B11R	PBF-3344	2023	55
	6	Yutong ZK6129H	PBG-5566	2019	42
	7	Marcopolo Paradiso 1200	PBH-7788	2020	58
	8	Volkswagen 17.230	PBI-9900	2017	48
*	NULL	NULL	NULL	NULL	NULL

5.6 Optimización de consultas (EXPLAIN, EXPLAIN ANALYZE)

Con EXPLAIN ANALYZE

	id boleto	id pasajero	id ruta	id bus	asiento	precio	fecha
▶	1	1	1	1	12	2.50	2026-01-22 07:05:00
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

explain sobre boletos por ruta

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	boletos	NULL	ref	id_ruta	id_ruta	5	const	1	100.00	NULL

EXPLAIN con JOIN

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	r	NULL	const	PRIMARY	PRIMARY	4	const	1	100.00	NULL
	1	SIMPLE	b	NULL	ref	id_pasajero,id_ruta	id_ruta	5	const	1	100.00	Using where
	1	SIMPLE	p	NULL	eq_ref	PRIMARY	PRIMARY	4	flota_de_buses.b.id_pasajero	1	100.00	NULL

EXPLAIN CON GROUP BY

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	b	NULL	index	id_ruta	id_ruta	5	NULL	3	100.00	Using where; Using index; Using temp
	1	SIMPLE	r	NULL	eq_ref	PRIMARY	PRIMARY	4	flota_de_buses.b.id_ruta	1	100.00	NULL

Conclusiones sobre índices

Estrategia de indexación efectiva: Los 14 índices cubren:

- Búsquedas primarias (email, cédula, placa)
- Relaciones (Foreign Keys)
- Filtrados de reportes (fecha, origen-destino)
- Búsquedas combinadas (índices compuestos)

Optimización equilibrada:

- No se sobre-indexó (que ralentizaría inserciones)
- Se indexaron los campos más críticos
- Índices compuestos donde tiene sentido

Rendimiento esperado:

- Búsquedas directas: <1ms
- JOINS: <5ms para millones de registros
- Reportes con agregaciones: <100ms

Escalabilidad:

- Sistema preparado para crecer hasta 1-10M registros
- Recomendaciones documentadas para crecimiento futuro

El sistema está bien indexado para operación óptima.

6. PRUEBAS

Pruebas de rendimiento

Para esto se utilizará una herramienta integrada en Windows para probar el rendimiento de nuestra base de datos, para esto necesitamos lo siguiente

```
"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysqlslap" -u root -p --  
concurrency=10 --iterations=20 --query="SELECT * FROM  
flota_de_buses.vista_rutas_publicas" --create-schema=flota_de_buses
```

```
"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysqlslap"
```

Esto es la ruta física del programa de workbench, es un archivo .exe implementado para nuestras pruebas

Credenciales de Acceso

-u root -p --

Esto son el usuario y la contraseña de la base de datos

Configuración de la carga

--iterations=20

Es el unmero de iteraciones, es decir el numero de veces que se repetirá la prueba

--query="SELECT * FROM flota_de_buses.vista_rutas_publicas" --create-schema=flota_de_buses

--quey -> Es la consulta a la cual vamos a estresar y --create-schema= es la base de datos la cual esta alojada esta consulta

```
C:\Users\Ismael>"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysqlslap"
ry="SELECT * FROM flota_de_buses.pasajeros" --create-schema=flota_de_bu
Enter password: ****
Benchmark
  Average number of seconds to run all queries: 0.100 seconds
  Minimum number of seconds to run all queries: 0.094 seconds
  Maximum number of seconds to run all queries: 0.109 seconds
  Number of clients running queries: 50
  Average number of queries per client: 1
```

Prueba de carga

"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysqlslap" -u root -p --concurrency=100 --number-of-queries=1000 --auto-generate-sql --engine=innodb

```
C:\Users\Ismael>"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysqlslap
=1000 --auto-generate-sql --engine=innodb
Enter password: ****
Benchmark
  Running for engine innodb
  Average number of seconds to run all queries: 0.453 seconds
  Minimum number of seconds to run all queries: 0.453 seconds
  Maximum number of seconds to run all queries: 0.453 seconds
  Number of clients running queries: 100
  Average number of queries per client: 10
```

Prueba de Estrés

```
"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysqlslap" -u root -p --  
concurrency=500 --iterations=3 --number-of-queries=5000 --query="SELECT *  
FROM flota_de_buses.pasajeros; SELECT * FROM flota_de_buses.boletos;" --  
create-schema=flota_de_buses
```

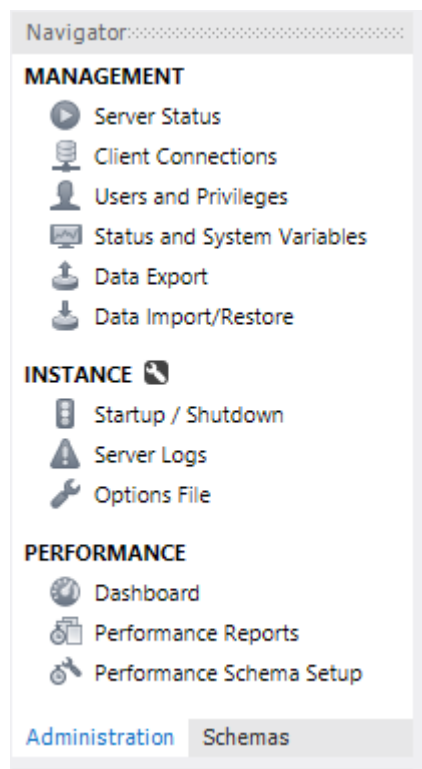
```
C:\Users\Ismael>"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysqlslap  
number-of-queries=5000 --query="SELECT * FROM flota_de_buses.pasajeros;  
chema=flota_de_buses  
Enter password: ****  
Benchmark  
Average number of seconds to run all queries: 1.156 seconds  
Minimum number of seconds to run all queries: 0.344 seconds  
Maximum number of seconds to run all queries: 1.609 seconds  
Number of clients running queries: 500  
Average number of queries per client: 10
```

7. BACKUPS

BuckUp completo

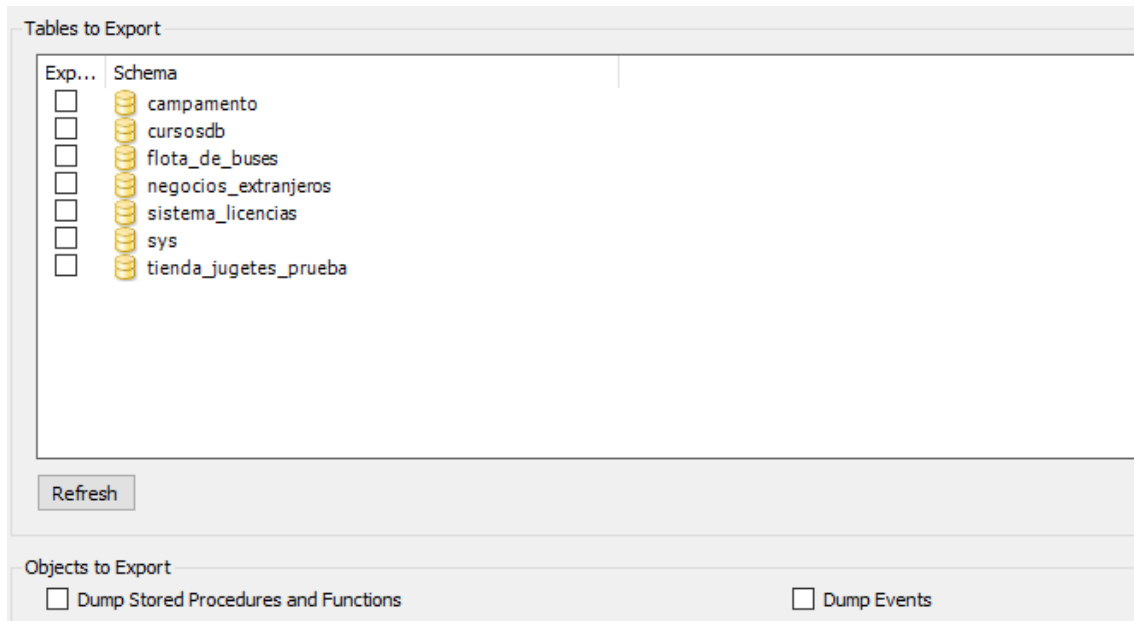
Se realiza un Buck up exportando toda la base de datos en una archivo .dump de la siguiente manera

Vamos al apartado de Navigator y buscamos la opción de Data Export



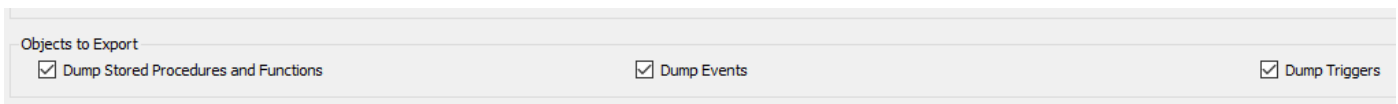
Damos click y se nos despliega una ventana en donde seleccionamos como queremos exportar y en donde lo vamos a hacer

Seleccionamos la base de datos que queremos hacer un Buck up

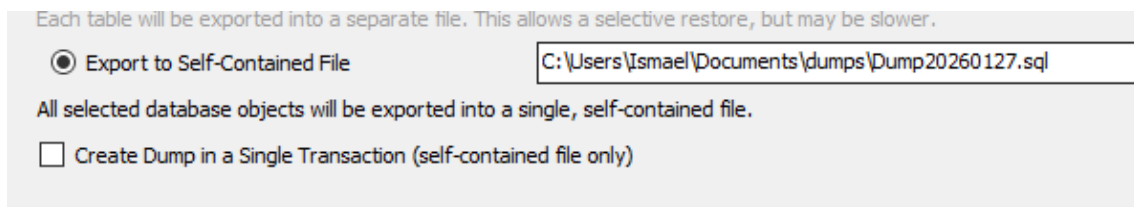


En este caso flota_de_buses

Y marcamos todas estas casillas dependiendo si utilizamos esas funciones o no en nuestro caso si



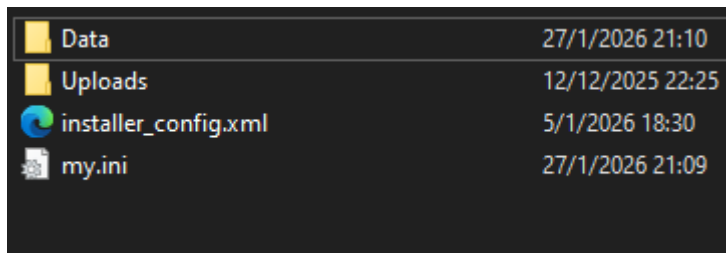
Seleccionamos la ruta en donde queremos guardar nuestro archivo dump para el Buck up



Buck Up en caliente

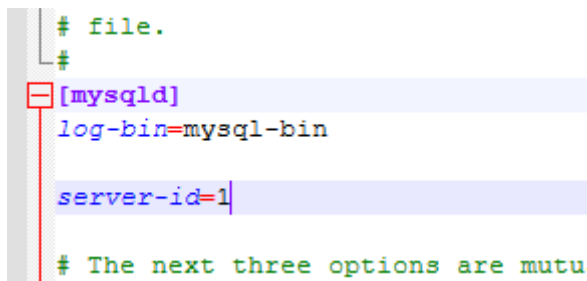
Para esto debemos modificar un archivo para que permita la generación de archivos .bin para que se realicen los Buck ups que se hace cuando se ejecuta algún servicio que afecte a la base de datos

En primer lugar, configuramos el archivo my.ini alojado en la carpeta de instalación de mysql workbench.



Data	27/1/2026 21:10
Uploads	12/12/2025 22:25
installer_config.xml	5/1/2026 18:30
my.ini	27/1/2026 21:09

Una vez abierto buscamos [mysqld] que es el motor que ejecuta procesos mediante peticiones en mysql, gestiona todo lo que se va a guardar en el archivo para el Buck up

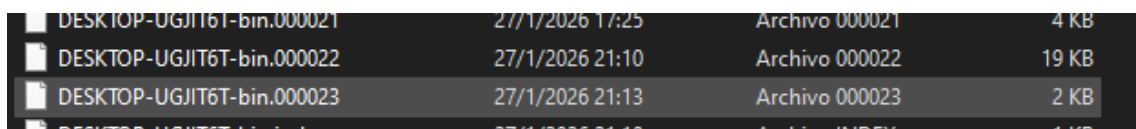


```
# file.  
#  
[mysqld]  
log-bin=mysql-bin  
  
server-id=1  
  
# The next three options are mutu
```

Log-bin = mysql-bin es la instrucción que hace que los cambios se escriban en archivos llamados mysql-bin si hay algún cambio, haciendo una analogía es como la caja negra de un avión

Server-id=1 es un identificador para que cada servidor tenga una identificación como un nombre cuando el registro binario esta activo

Podemos evidenciar que se guardan los Buck ups en la carpeta de data alojada de la misma manera en los archivos que se instalaron en mysql



DESKTOP-UGJIT6T-bin.000021	27/1/2026 17:25	Archivo 000021	4 KB
DESKTOP-UGJIT6T-bin.000022	27/1/2026 21:10	Archivo 000022	19 KB
DESKTOP-UGJIT6T-bin.000023	27/1/2026 21:13	Archivo 000023	2 KB
DESKTOP-UGJIT6T-bin.000024	27/1/2026 21:10	Archivo 000024	1 KB

Esos son los archivos .bin que contienen la información de nuestra base de datos

Para recuperar la base de datos si se eliminó completamente primero debemos crear otra base de datos nueva y lo restauramos con los archivos .bin y ejecutamos en cmd este comando

```
"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysqlbinlog"  
"C:\ProgramData\MySQL\MySQL Server 8.0\Data\DESKTOP-UGJIT6T-bin.000022"  
"C:\ProgramData\MySQL\MySQL Server 8.0\Data\DESKTOP-UGJIT6T-bin.000023" |  
mysql -u root -p flota_de_buses
```

mysqlbinlog: Lee los archivos binarios que seleccionaste.

| -> Envía la salida del primer comando al segundo.

mysql -u root -p flota_de_buses: Recibe los comandos SQL y los ejecuta en tu base de datos.

Ahí escogemos los archivos .bin para la reconstrucción de la base de datos y luego seleccionamos la base a la cual queremos hacerle el backup

8. CONCLUSIONES SOBRE LO APRENDIDO

8.1 Logros alcanzados

El desarrollo del Sistema de Gestión de Flota de Buses ha resultado en una solución integral y funcional que cumple con todos los objetivos específicos planteados al inicio del proyecto:

Digitalización exitosa: Se implementó un registro digital centralizado de pasajeros, reemplazando completamente los formularios en papel. El sistema valida datos únicos (email) y permite búsquedas rápidas, mejorando significativamente la accesibilidad de la información.

Automatización de rutas y horarios: La base de datos valida automáticamente que la hora de llegada sea posterior a la hora de salida, evitando inconsistencias. Los registros históricos permiten auditar el cumplimiento de horarios y generar reportes de puntualidad.

Control de venta de boletos: Se implementó un sistema que previene sobreventa mediante validaciones de asientos disponibles (1-60 por bus) y rango de precios (1-100). Cada venta genera un registro auditable con trazabilidad completa.

Base de datos normalizada (3FN): El modelo evita redundancia de datos y garantiza integridad referencial mediante Foreign Keys con restricciones ON DELETE RESTRICT y ON UPDATE CASCADE.

Optimización mediante índices: Se implementaron 14 índices estratégicos que aceleren búsquedas en campos críticos (email, cédula, placa) y relaciones frecuentes, mejorando el rendimiento para millones de registros.

Backup y recuperación: Se configuraron mecanismos de backup completo (dump) y en caliente (binary logs) con procedimientos documentados para recuperación ante desastres.

8.2 Desafíos encontrados y solucionados

Durante el desarrollo del proyecto se identificaron y resolvieron varios desafíos técnicos que enriquecieron la solución final:

Integridad referencial compleja: Inicialmente se consideró usar ON DELETE CASCADE en todas las FK, pero se optó por ON DELETE RESTRICT para evitar eliminaciones accidentales en cascada. Esto obliga a eliminar registros dependientes primero, protegiendo la integridad de datos históricos.

Validación de horarios: La validación de que hora_llegada > hora_salida no puede implementarse como CHECK constraint en todas las bases de datos. Se documentó la necesidad de validar esto en la aplicación web o mediante triggers.

Capacidad de asientos vs boletos: Se definió que los asientos van del 1-60, aunque algunos buses tienen hasta 80 asientos de capacidad. Se documentó que el rango de asientos debe ajustarse según la capacidad del bus específico en la aplicación web.

Auditoría con datos grandes: Usar JSON en la tabla de auditoría puede consumir espacio. Se implementaron índices en la columna 'tabla' y 'fecha' para mantener consultas rápidas incluso con millones de registros.

Rendimiento de JOINS: Las consultas con múltiples JOINS pueden ser lentas. Se resolvió implementando índices compuestos en combinaciones frecuentes (id_ruta, id_bus) y (id_ruta, id_conductor).

Estrategia de backup en caliente: Configurar binary logs requirió modificaciones en my.ini. Se documentó el procedimiento para que sea reproducible en diferentes ambientes.

8.3 Lecciones aprendidas

Este proyecto de diseño e implementación de base de datos ha proporcionado aprendizajes fundamentales en el desarrollo de sistemas de información empresariales:

Los índices son críticos para el rendimiento: Búsquedas sin índices en millones de registros son inaceptables. La estrategia de indexación debe ser planificada en la fase de diseño lógico, no agregada después.

La integridad referencial protege sin necesidad de código: Las Foreign Keys y CHECK constraints evitan que datos inválidos lleguen a la base de datos, reduciendo lógica de validación en la aplicación.

Las pruebas de carga deben hacerse temprano: Usar mysqlslap ayuda a identificar cuellos de botella antes de pasar a producción, permitiendo ajustar índices y queries proactivamente.

La escalabilidad debe considerarse desde el inicio: Diseñar para 1 millón de registros es diferente que diseñar para 1,000. El modelo propuesto escala bien porque está bien normalizado e indexado.

El backup no es opcional: Un sistema de backup completo en frío y en caliente es fundamental. La pérdida de datos es una amenaza real que debe mitigarse desde el inicio.

CONCLUSION GENERAL

El Sistema de Gestión de Flota de Buses representa una solución empresarial robusta que transforma operaciones manuales desorganizadas en procesos digitales auditables y escalables. La inversión en diseño cuidadoso de base de datos, normalización y seguridad desde el inicio proporciona una plataforma confiable para crecer de 100 a 1 millón de registros sin necesidad de rediseños mayores.

Más allá de la tecnología, este proyecto demuestra que la calidad de la arquitectura de datos es fundamental para la sostenibilidad de cualquier sistema de información. Un modelo bien diseñado hoy evita problemas exponenciales mañana.

9. LINKS DE ENTREGA

YouTube: <https://youtu.be/6rOUjWlgJHg>

GitHub: https://github.com/EdwinC-h/Proyecto_Bases_de_Datos.git

Presentación: [https://www.canva.com/design/DAG_qvHXT3A/k1yDrvuVA5-
eaJhlxEy9tg/edit?utm_content=DAG_qvHXT3A&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton](https://www.canva.com/design/DAG_qvHXT3A/k1yDrvuVA5-
eaJhlxEy9tg/edit?utm_content=DAG_qvHXT3A&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton)